

Project 2 实验报告：光照模型与光线追踪

任务 3：光线追踪与阴影投射

1.1 实现原理

1.1.1 递归光线追踪

光线追踪的核心在于模拟光线在物体表面的多次反射。当光线击中具有镜面反射属性（Specular）的材质时，我们根据法线 N 和入射方向（视线反方向） V 计算反射方向 R ：

$$R = 2(N \cdot V)N - V$$

通过递归调用 `traceRay`，我们可以获取反射路径上的颜色。最终像素颜色的计算公式为：

$$I_{total} = I_{ambient} + I_{direct} + k_s \times I_{reflected}$$



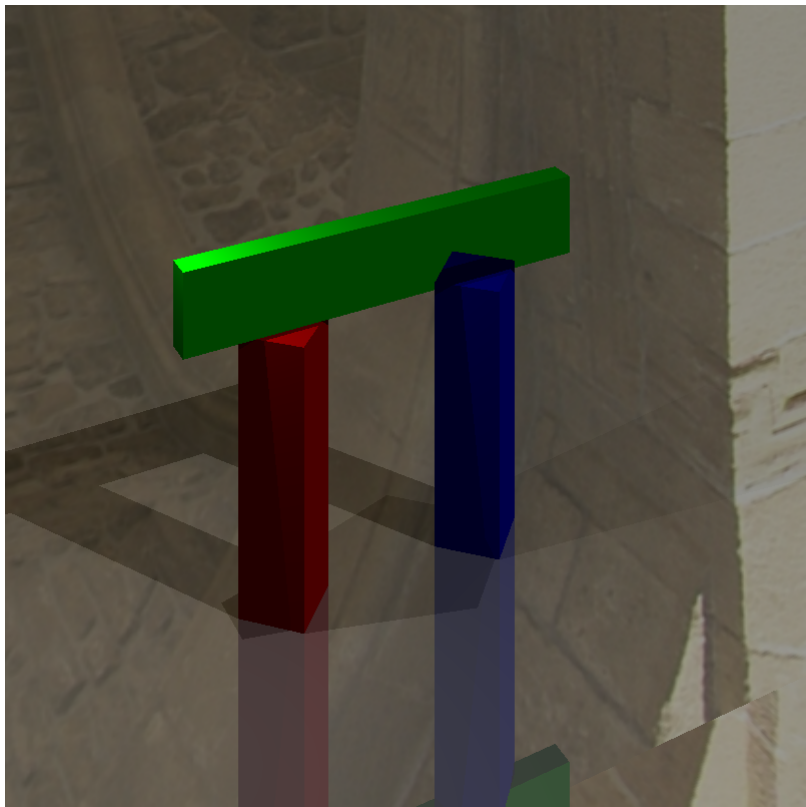
1.1.2 阴影投射

为了确定点光源是否能照亮命中点，我们从命中点向光源发射一根 **阴影射线**。

- **自相交处理**：为了防止由于浮点数精度导致的“自遮挡”现象，射线起点需沿法线方向进行微小偏移（即代码中的 `0.001f`）。
- **遮挡判定**：若阴影射线在 $t \in [0.001, \text{distToLight}]$ 范围内检测到交点，则该点处于阴影中，不计算直接光照。

1.2 关键代码实现 (Renderer.cpp)

```
1 // 简化版核心逻辑展示
2 Vector3f Renderer::traceRay(const Ray &r, float tmin, int bounces, Hit &h)
3 const {
4     if (!_scene.getGroup()->intersect(r, tmin, h))
5         return _scene.getBackgroundColor(r.getDirection());
6
7     Material* mat = h.getMaterial();
8     Vector3f N = h.getNormal().normalized();
9     Vector3f hitPoint = r.pointAtParameter(h.t);
10
11     // 1. 环境光
12     Vector3f ambient = _scene.getAmbientLight() * mat->getAmbientColor();
13     Vector3f directLight(0, 0, 0);
14
15     // 2. 直接光照与阴影
16     for (int i = 0; i < _scene.getNumLights(); ++i) {
17         Vector3f dirToLight, lightIntensity;
18         float distToLight;
19         _scene.getLight(i)->getIllumination(hitPoint, dirToLight,
20         lightIntensity, distToLight);
21
22         Ray shadowRay(hitPoint + dirToLight * 0.001f, dirToLight);
23         Hit shadowHit;
24         if (!_args.shadows || !_scene.getGroup()->intersect(shadowRay,
25         0.001f, shadowHit) || shadowHit.getT() > distToLight) {
26             directLight += mat->shade(r, h, dirToLight, lightIntensity);
27         }
28     }
29
30     // 3. 递归反射 (Indirect Light)
31     Vector3f indirectLight(0, 0, 0);
32     if (bounces > 0 && mat->getSpecularColor().length() > 0.001f) {
33         Vector3f v = -r.getDirection().normalized();
34         Vector3f R = (2.0f * Vector3f::dot(N, v) * N - v).normalized();
35         Ray reflectedRay(hitPoint + N * 0.001f, R);
36         Hit reflectedHit;
37         indirectLight = traceRay(reflectedRay, 0.001f, bounces - 1,
38         reflectedHit) * mat->getSpecularColor();
39     }
40
41     return ambient + directLight + indirectLight;
42 }
```



拓展任务：抗锯齿

2.1 实现原理

由于每个像素只采集中心点的信息，当场景中存在高频几何边缘时，会产生阶梯状的 **锯齿**。

2.1.1 抖动采样

我们不再固定抓取像素中心，而是在像素区域内引入随机偏移，进行多次采样取平均。这种方法将锯齿转化成了人眼较易接受的噪声。

2.1.2 高斯滤波

为了进一步提升平滑度，本实验采用了 **超采样 (Supersampling)** 结合 **高斯平滑** 的方案：

1. **3倍上采样**：将 300×300 的图像渲染为 900×900 。
2. **加权卷积**：使用 3×3 高斯核对像素及其邻域进行加权。
3. **下采样**：将过滤后的结果归一化并回填至原始尺寸。

2.2 抗锯齿效果对比

参数	渲染时间	视觉效果
基础渲染	~5s	边缘锯齿明显，反射不连续
-jitter (16 samples)	~80s	锯齿大幅减少，边缘出现轻微柔化
-jitter -filter	~240s	边缘极度平滑，光影过渡自然，画质类似照片

2.2.1渲染场景 6 兔子







(从上至下：含阴影基础渲染、仅抖动渲染、抗锯齿+抖动渲染)

三、总结

本实验完成了包含多级 *bounces* 递归反射、硬阴影投射在内的核心任务，并成功实现了抖动采样与高斯滤波等高质量抗锯齿拓展功能。在技术实施层面，我们通过引入 $\epsilon = 0.001$ 的射线起点偏移，显著提升了数值稳定性并有效消除了自相交产生的黑斑；面对抗锯齿处理带来的巨大计算压力，明确了通过 OpenMP 循环并行化来平衡性能的优化路径；同时，所有采样平均处理均坚持在线性空间内完成，从底层逻辑上保证了物理渲染的准确性。